Internship Project – Data Processing and Feature Extraction Pipeline

Table of Contents

1. Introduction

In today's world, everything becomes online and reachable through internet with one click away. As a result of that, e-commerce is an improving field in technology with an inevitably fast pace. In this fast revolution of e-commerce, retailers need to improve themselves continuously. Hence, they are increasingly using varied machine learning techniques to find a solution to various problems of this field. This project focuses on the problem of shopping cart abandonment and purchase intent prediction. It aims to predict user's intent and future behavior from previous actions.

In my part of the project, we were assigned to a job which we need to convert the given implementation of feature extraction pipeline, to make it scalable using Apache Spark for big data analysis.

2. Project Background

a. Status of the project and problem definition/ motivation

Initially, we were given the implementation of feature extraction code and a small part of the data. Pipeline had 12 functions and 93 features & labels which are already implemented. The data originally consists of ~130 million user actions covering several months and we had 13 million of that to test our final implementation. Initial implementation of the code was designed to run in a single machine with a much smaller data. Our job was to develop a scalable implementation which can run 130 million action at once, in a half time of the initial implementation. It was expected to run the scalable code in two machines at the same time by distributing the data.

3. Internship Project

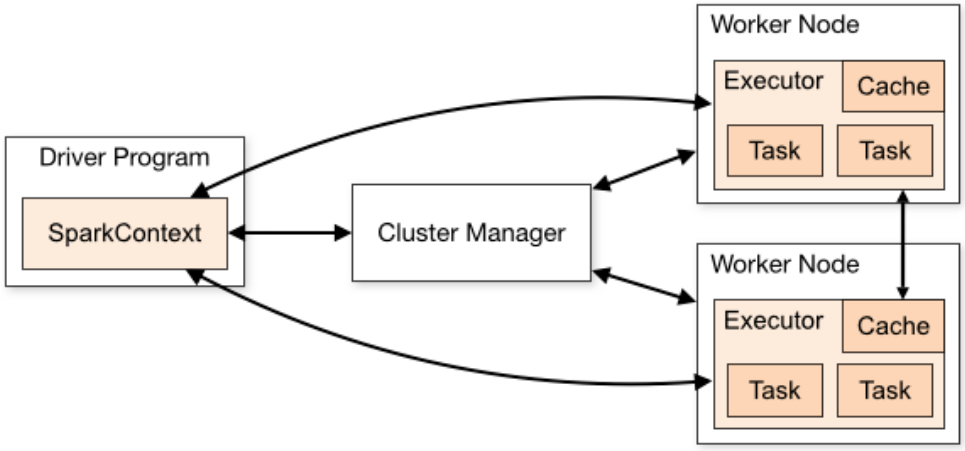
a. Project Objective

Main purpose of this step of the project is to prepare the data in a scalable way. In order to process the data in a machine learning model we need to extract features to have more meaningful and useful results. We already had implemented features which we would re-implement them in order to run on two servers simultaneously using Apache Spark (python) in the standalone spark cluster.

b. Methodology/tool

The methodologies/tools that we used are listed below:

- Jupyter: The notebook platform for editing the code, can be used through anaconda application.
- Apache Spark: It was the main tool for this project. It is a distributed framework that can handle big data analysis. It used for distributing data processing tasks across multiple computers within a cluster.
- Pyspark
    - Python API for Apache Spark
    - RDD (resilient distributed dataset): Fault-tolerant and distributed datasets. Two types of data operations can be operated: transformations and actions.
        - Transformation: Creates a new dataset from existing one
        - Action: Returns a value to the driver program after running a computation on the dataset
    - Cluster mode: Standalone cluster manager
        - Spark applications run as independent sets of processes on a cluster, coordinated by the SparkContext (*from pyspark import SparkContext*) object in the driver program.
        - Spark context connects to cluster manager and sends tasks to executors to run.



c. Project Details

- Installation of spark and anaconda environment
- Re-implementation of the given code:
    - Binning by session id, user id and sort by timestamp: user ids are unique for each user. When a user enters the website a session id is created at each time. During every session user makes multiple actions, for each action a row is created in our input dataset. We need to keep together the sessions which belong to each user and sort them according to date.
    - Re-implementing the feature functions for them to be suitable with RDD

- When the code is ready, firstly we tried the code locally in a spark session without running on the cluster mode
- Comparing our outputs with expected outputs and checking whether they match
- Running the spark standalone cluster mode and deploying master&slave machines: Details of the cluster mode are explained on readme.md step by step
- Result: Execution of the code for 13 million action rows took approximately 5 minutes in cluster mode deployed on 1 master and 1 worker (with 3 cores).